

INFO 408 Assignment 2

Name: Kwok Hoi Yi, Any (8271933)

A Short Technical Report – Implementation of a Large-scale Scalable Song Database with MongoDB

Introduction

In today's digital world, data is everywhere. Especially in the music industry, where millions of songs, artists, and user preferences are stored and accessed every day. Managing such a huge amount of music data isn't easy, especially when it includes things like emotions, tempo, genres, lyrics, and user-related tags. Traditional relational databases (like SQL) are often too rigid to handle this kind of semi-structured or nested data. For example, some songs may have multiple artists, missing release dates, or extra fields like "similar songs." These make it hard to fit everything neatly into rows and columns. So, we need a more flexible and scalable solution.

This project is to design and build a song database using MongoDB, a NoSQL database that works well with messy or complex data structures. The aim is to create a backend database that supports music player applications needing quick access to large collections of songs, along with metadata, recommendation tags, and filters for different features.

Background Information

Dataset Overview

The dataset used in this project is titled " 900K+ Spotify Songs with Lyrics, Emotions & More", sourced from Kaggle in JSON format. It was a supporting of the Abracadabra project (part of the Top 200 projects in the NVIDIA Llama-Index Contest), a Retrieval-Augmented Generation (RAG) system designed for intelligent playlist generation using large language models (LLMs). (Kaggle, n.d.) The uncompressed file is approximately 1.3 GB in size and contains 498,052 records. Each record represents a single Spotify track and includes basic information such as title, artist(s), lyrics, album, genre, emotion, duration, release date, and musical features like tempo, key, time signature, loudness, and popularity. It also provides numerical scores for features like energy, danceability, and positiveness, along with various boolean contextual tags ("Good for Party", "Good for Work/Study", "Good for Relaxation/Meditation", "Good for Exercise", "Good for Running", "Good for Yoga/Stretching", "Good for Driving", "Good for Social Gatherings", "Good for Morning Routine"). Each song

may also contain up to three similar songs stored in a nested structure (Appendix 1 Raw Data).

Due to its large size and semi-structured, and nested format, this dataset is not well-suited for storage in a flat relational model like SQL. Storing it in a relational schema would require extensive preprocessing and normalization, such as splitting artists, audio features, and similar songs into separate tables connected by foreign keys. While this approach might improve data integrity, it would also significantly complicate query design, especially for aggregation tasks involving related fields. Moreover, the target usage, a music player which only requires relatively simple queries such as filtering by artist or genre. There's no need to over-engineer the database structure for such use cases.

In addition, the flexibility needed to handle irregular, optional, or missing fields is difficult to maintain in traditional relational databases without frequent schema modifications. While graph databases excel at modelling explicit relationships (e.g., artist collaborations, user-song interactions), such relationships are not well-defined in this dataset too. By contrast, this dataset is song-centric, focusing on individual songs and their metadata rather than on artist-to-artist relationships.

The Reason of choosing MongoDB

Given the semi-structured nature of the raw data, the need for flexibility in handling null or missing values, the presence of nested relationships (e.g., similar songs), and the relatively simple and straightforward query usage, MongoDB was chosen for this implementation. MongoDB is a document-oriented NoSQL database that allows each song to be stored as a self-contained document, preserving its nested structure and supporting variable fields across different records.

Assuming the dataset is intended for use in a music player app where queries are typically straightforward and rarely involve complex joins, MongoDB helps avoid the complexity of implementing a rigid relational schema. It eliminates the need to split data across multiple tables and perform costly joins, making it suitable for applications that prioritize performance and simplicity.

In contrast to SQL databases, MongoDB's flexible, schema-less model is better suited for data where fields may be optional or inconsistently populated. This flexibility allows developers to adapt the data model over time without costly migrations or downtime. MongoDB also supports indexing on arrays and nested fields, enabling efficient filtering on common fields such as genre, artist, or contextual tags.

Additionally, MongoDB is highly scalable and handles large datasets efficiently, especially when appropriate indexes are applied. It integrates seamlessly with Python via the PyMongo library, simplifying data loading, transformation, and insertion operations. This was particularly useful in this project, as the raw dataset was cleaned and restructured entirely in Python before being inserted into MongoDB.

Database Implementation

Data Pre-processing and Transformation

Due to the large size of the raw dataset, we first extracted a sample of around 5,000 records for easier handling and initial design. Specifically, we selected one record every 500 entries to ensure a variety of artists with different names and create a manageable subset for testing. This approach allowed for easier cleaning, validation, and the development of transformation logic and schema design without being slowed down or confused by the full dataset. Once the data structure and indexing strategy were finalized, the same logic was applied to the complete dataset through the same Python script.

To facilitate efficient data handling and enable reusability for full dataset processing, the entire implementation was done in Python, using relevant libraries to clean the data and import it into MongoDB. The raw JSON dataset was first loaded into Python, then parsed line by line. Each line represents one JSON object (a song), which was converted into a Python dictionary and appended to a list.

After loading, a custom cleaning function (Appendix 2 Python cleaning function script) was applied to standardize formats and consolidate related fields. Several transformations were made to improve structure and enable more accurate filtering and indexing:

1. The Artist(s) field, originally stored as a single string, was split by commas and stored as a list of artists.
2. The Feature Score fields were grouped into a nested dictionary under a new field called Feature_Scores.
3. The boolean contextual tags were dynamically filtered (only those with a true value were retained) and converted into a list of applicable tags.
4. The "Similar Songs" subfields were reformatted into a list of dictionaries, each containing the structure of {"Artist": ..., "Title": ..., "Similarity Score": ...}.
5. To keep the simplicity of the dataset, we dropped the Emotion, Key, Tempo, Loudness (db), Time Signature, and Explicit fields.

This restructuring supports cleaner data representation and improves MongoDB's ability to perform effective queries and indexing.

Structure Design

MongoDB does not require a strict schema, so we designed the data store in our customized structure. The structure was designed to reflect the natural format of the dataset while optimizing for common use cases, such as searching by artist or genre. MongoDB allows each song to be represented as a self-contained document where all relevant metadata is stored together. This eliminates the need for table joins and keeps related data in one place for fast and direct access.

The final structure for each song document includes fields directly derived from the raw data (Title, Lyrics, Length, Genre, Album, Release_Date, and Popularity), multi-valued fields (lists) such as Artists and Contextual_Tags, and nested documents like Feature_Scores and Similar_Songs. (Appendix 3 Songs collection sample)

This design supports efficient filtering and searching on fields with multiple values. Crucially, the schema was designed with no assumption of completeness. Missing fields are common in real-world datasets, and in our data set. MongoDB's flexible document model allows such incomplete records to coexist without issues.

With appropriate indexing, this schema provides efficient query performance for typical music application use cases, such as filtering songs by artist, album, or contextual tags.

Database Implementation

All data loading, cleaning, transformation, and database operations were performed in Python using the related libraries pymongo and json. The implementation script was modular, with separate sections for file loading, record cleaning, MongoDB insertion, and index creation. After processing, the cleaned dataset was inserted into a with a collection named songs under the database spotify_db. Before each insertion, the existing collection was dropped to ensure data consistency and avoid duplicates. (Appendix 4 Python import script) Once the data was inserted, indexes were created using MongoDB's create_index() method on relevant fields

Indexing Strategy and Query Optimization

Given the size of the dataset, indexing is critical for maintaining query performance. We created indexes on the most commonly queried fields: Artists, Genre, Album and Contextual_Tags. MongoDB supports indexing on arrays and nested fields, allowing complex queries to remain performant. While compound or multikey indexes can further optimize specific queries, we intentionally chose to apply single-field indexes at this stage to strike a balance between query speed and index maintenance overhead. Indexes can slow down insert and update operations due to the additional write cost, but given that music applications are typically read-heavy, this trade-off is acceptable. If more complex querying becomes a future requirement, MongoDB's flexibility will allow us to add compound or multikey indexes as needed without major changes to the existing setup.

This Python-based implementation ensures repeatability and scalability. If the dataset needs to be updated or expanded, the same script can be reused with minimal modification.

(Appendix 5 Python indexing script, Query result)

Challenging

During the implementation process, several issues were encountered:

1. Initial Import via Mongo Shell Was Not Repeatable and Hard to Clean Data

We first attempted to use mongoimport via the MongoDB shell to load the dataset directly. While simple, this method was not ideal for repeated runs or when data cleaning was required. It was also difficult to customize and unsuitable for handling nested fields or structural transformations. Switching to Python (using libraries json and pymongo) allowed us to build a more controlled and reusable pipeline, enabling more efficient data cleaning.

2. Sampling for Early Design

Due to the large size and messy structure of the dataset, it was difficult to get a clear overview during initial exploration. To address this, we decided to take a representative sample for design and testing. Specifically, we selected one record every 500 entries, resulting in a sample of approximately 5,000 songs. This approach provided broader coverage of the dataset's diversity and made it easier to clean and validate the schema without being slowed down by the full dataset.

3. Inconsistency of the Artists Data

The raw data's artist information is stored in inconsistent formats, mixing group names (often enclosed in quotation marks), individual artist names, and some unclear or suspicious entries. We removed the quotation marks from group names and treated them as artist names, as maintaining a flat structure improves efficiency. Given MongoDB's scalability, restructuring nested artist groups later, if necessary, would not be overly complex. Conversely, uncertain or suspicious names were retained to avoid premature data loss.

Conclusion

This project demonstrates the practical application of MongoDB for managing a large, semi-structured music dataset. By leveraging MongoDB's flexible document model, we were able to accommodate inconsistent, nested, and optional fields without the complexity of relational normalization. This approach simplified data storage, reduced schema overhead, and enabled straightforward querying which are crucial for music applications focused on filtering and metadata retrieval.

Compared to relational databases, MongoDB avoided the need for rigid schemas and costly joins. It also offered a simpler and more efficient way to store loosely connected data like "similar songs," which did not justify a full relationship model while compared to graph databases.

Overall, MongoDB strikes a practical balance between flexibility, scalability, and ease of use. These characteristics which prioritized rapid access and adaptability to evolving data made it the choice for this project. With the indexing strategy in place, the system can scale further to support additional features without major architectural changes.

Disclosure of Use of Large Language Models

In this assignment, I used ChatGPT for the following purposes:

- Assisting with python code
- Checking and improving the clarity of my English

References

GeeksforGeeks. (2022, June 03). *Working with JSON data in Python*.

<https://www.geeksforgeeks.org/working-with-json-data-in-python/>

Kaggle. (n.d.).  900K+ Spotify Songs with Lyrics, Emotions & More.

<https://www.kaggle.com/datasets/maharshipandya/spotify-spotify-lyrics-dataset>

MongoDB Inc. (n.d.). *MongoDB Manual*. <https://www.mongodb.com/docs/manual/indexes/>

MongoDB Inc. (n.d.). *PyMongo 4.7 Documentation*.

<https://pymongo.readthedocs.io/en/stable/>

Tofighi F. (2024 Feb 10). *MongoDB indexing tutorial with examples*. Medium.

<https://farshadth.medium.com/mongodb-indexing-tutorial-with-examples-9b3534e9b422>

Appendix

Appendix 1 Raw data

```
▼ "root" : { 31 items
  "Artist(s)" : string "!!!"
  "song" : string "Even When the Waters Cold"
  "text" :
    string "Friends told her she was better off at the bottom of a river Than in a bed with him He said "Until you try both, you won't know what you like better Why don't we go for a swim?" Well, friends told her this and friends told her that But friends don't choose what echoes in your head When she got bored with all the idle chit-and-chat Kept thinking 'bout what he said I'll swim even when the water's cold That's the one thing that I know Even when the water's cold She remembers it fondly, she doesn't remember it all But what she does, she sees clearly She lost his number, and he never called But all she really lost was an earring The other's in a box with others she has lost I wonder if she still hears me I'll swim even when the water's cold That's the one thing that I know Even when the water's cold If you believe in love You know that sometimes it isn't Do you believe in love? Then save the bullshit questions Sometimes it is and sometimes it isn't Sometimes it's just how the light hits their eyes Do you believe in love?"
  "Length" : string "03:47"
  "emotion" : string "sadness"
  "Genre" : string "hip hop"
  "Album" : string "Thriller"
  "Release Date" : string "2013-04-29"
  "Key" : string "D min"
  "Tempo" : float 0.4378698225
  "Loudness (db)" : float 0.785065407
  "Time signature" : string "4/4" ⓘ
  "Explicit" : string "No"
  "Popularity" : string "40"
  "Energy" : string "83"
  "Danceability" : string "71"
  "Positiveness" : string "87"
  "Speechiness" : string "4"
  "Liveness" : string "16"
  "Acousticness" : string "11"
  "Instrumentalness" : string "0"
  "Good for Party" : int 0
  "Good for Work/Study" : int 0
  "Good for Relaxation/Meditation" : int 0
  "Good for Exercise" : int 0
  "Good for Running" : int 0
  "Good for Yoga/Stretching" : int 0
  "Good for Driving" : int 0
  "Good for Social Gatherings" : int 0
  "Good for Morning Routine" : int 0
  ▼ "Similar Songs" : [ 3 items
    ▼ 0 : { 3 items ⓘ
      "Similar Artist 1" : string "Corey Smith" ⓘ
      "Similar Song 1" : string "If I Could Do It Again"
      "Similarity Score" : float 0.9860607848
    }
    ▼ 1 : { 3 items
      "Similar Artist 2" : string "Toby Keith"
      "Similar Song 2" : string "Drinks After Work"
      "Similarity Score" : float 0.9837194774
    }
    ▼ 2 : { 3 items
      "Similar Artist 3" : string "Space"
      "Similar Song 3" : string "Neighbourhood"
      "Similarity Score" : float 0.9832363508
    }
  ]
}
```

Appendix 2 Python cleaning function script

```
def clean_record(record):  
    clean_record={}  
  
    # Artists  
    raw_artists = record.get("Artist(s)", "")  
    split_artists = raw_artists.split(",") # Split by comma  
    # Clean each name and process  
    artists = []  
    for a in split_artists:  
        cleaned = a.strip().strip('"').strip("'") # Remove outer quotes and whitespace  
        if (  
            cleaned == "" or  
            cleaned.upper() in ["NA", "NULL", "UNKNOWN"] or  
            cleaned.count("?") >= 3  
        ):  
            cleaned = "Unknown"  
        artists.append(cleaned)  
    clean_record["Artists"] = list(dict.fromkeys(artists))  
  
    # Other fields  
    clean_record["Title"] = record.get("song")  
    clean_record["Lyrics"] = record.get("text")  
    clean_record["Length"] = record.get("Length")  
    clean_record["Genre"] = record.get("Genre")  
    clean_record["Album"] = record.get("Album")  
    clean_record["Release_Date"] = record.get("Release Date")  
    clean_record["Popularity"] = record.get("Popularity")  
  
    # Feature Score  
    score_fields = ["Energy", "Danceability", "Positiveness", "Speechiness",  
                    "Liveness", "Acousticness", "Instrumentalness"]  
    clean_record["Feature_Scores"] = {k: record.get(k, None) for k in score_fields}  
  
    # Contextual tags  
    clean_record["Contextual_Tags"] = [k for k in record.keys() if k.startswith("Good for") and record[k] == 1]  
  
    # Similar songs  
    similar_songs_raw = record.get("Similar Songs", [])  
    similar_songs = []  
    for group in record.get("Similar Songs", []):  
        for k in group.keys():  
            if k.startswith("Similar Artist"):  
                idx = k.split()[-1]  
                artist = group[k]  
                song_key = f"Similar Song {idx}"  
                title = group.get(song_key)  
                score = group.get("Similarity Score")  
                if artist and title:  
                    similar_songs.append({  
                        "Artist": artist,  
                        "Title": title,  
                        "Similarity Score": score  
                    })  
  
    clean_record["Similar_Songs"] = similar_songs  
    return clean_record
```


Appendix 3 Songs collection sample

```
Command Prompt - mongod
spotify_db> db.songs.findOne()
{
  _id: ObjectId('6825aefcd342b72435ff68f'),
  Artists: [ '!!!' ],
  Title: 'Even When the Waters Cold',
  Lyrics: 'Friends told her she was better off at the bottom of a river Than in a bed with him He said "Until you try both, you won\'t know what you like bet
ter Why don\'t we go for a swim?" Well, friends told her this and friends told her that But friends don\'t choose what echoes in your head When she got bored
with all the idle chit-and-chat Kept thinking \'bout what he said I\'ll swim even when the water\'s cold That\'s the one thing that I know Even when the water\'s
cold She remembers it fondly, she doesn\'t remember it all But what she does, she sees clearly She lost his number, and he never called But all she really l
ost was an earring The other\'s in a box with others she has lost I wonder if she still hears me I\'ll swim even when the water\'s cold That\'s the one thing th
at I know Even when the water\'s cold If you believe in love You know that sometimes it isn\'t Do you believe in love? Then save the bullshit questions Someti
mes it is and sometimes it isn\'t Sometimes it\'s just how the light hits their eyes Do you believe in love?',
  Length: '03:47',
  Genre: 'hip hop',
  Album: 'The!!!er',
  Release_Date: '2013-04-29',
  Popularity: '40',
  Feature_Scores: {
    Energy: '83',
    Danceability: '71',
    Positiveness: '87',
    Speechiness: '4',
    Liveness: '16',
    Acousticness: '11',
    Instrumentalness: '0'
  },
  Contextual_Tags: [],
  Similar_Songs: [
    {
      Artist: 'Corey Smith',
      Title: 'If I Could Do It Again',
      'Similarity Score': 0.9860607848
    },
    {
      Artist: 'Toby Keith',
      Title: 'Drinks After Work',
      'Similarity Score': 0.9837194774
    },
    {
      Artist: 'Space',
      Title: 'Neighbourhood',
      'Similarity Score': 0.9832363508
    }
  ]
}
```

Appendix 4 Python import script

```
# Import to MongoDB
db_name = "spotify_db"
collection_name = "songs"

client = MongoClient("mongodb://localhost:27017/")
db = client[db_name]
collection = db[collection_name]

collection.drop()
collection.insert_many(cleaned)
```

Appendix 5 Python indexing script

```
# Index
indexes = [
    {"Artists": 1},
    {"Genre": 1},
    {"Album": 1},
    {"Contextual Tags": 1}
]

for idx in indexes:
    collection.create_index(idx)
```

Appendix 5 Query Result

```
mongosh mongodb://127.0.0.1 x + v
switched to db spotify_db
spotify_db> show collections
songs
spotify_db> db.songs.find({ "Artists": "Taylor Swift" }).limit(1)
[
  {
    _id: ObjectId('6825aeffd342b724360a95d'),
    Artists: [ 'Big Red Machine', 'Taylor Swift' ],
    Title: 'Renegade',
    Lyrics: '[Verse 1: Taylor Swift] I tapped on your window on your darkest night The shape of you was jagged and weak There was nowhere for me to stay, but I stayed anyway And if I would\'ve known how many pieces you had crumbled into I might have let them lay [Chorus: Taylor Swift with Justin Vernon] Are you really gonna talk about timing in times like these? And let all your damage damage me And carry your baggage up my street And make me your future history, it\'s time You\'ve come a long way; open the blinds, let me see your face You wouldn\'t be the first renegade to need somebody Is it insensitive for me to say, "Get your shit together so I can love you"? Is it really your anxiety that stops you from giving me everything? Or do you just not want to? [Verse 2: Taylor Swift] I tapped on your window on your darkest night The shape of you was jagged and weak There was nowhere for me to stay, but I stayed anyway You fire of f missiles \'cause you hate yourself But do you know you\'re demolishing me? And then you squeeze my hand as I\'m about to leave [Chorus: Taylor Swift with Justin Vernon] Are you really gonna talk about timing in times like these? And let all your damage damage me And carry your baggage up my street And make me your future history, it\'s time You\'ve come a long way; open the blinds, let me see your face You wouldn\'t be the first renegade to need somebody Is it insensitive for me to say, "Get your shit together so I can love you"? Is it really your anxiety that stops you from giving me everything? Or do you just not want to? [Bridge: Taylor Swift] And if I would\'ve known How sharp the pieces were you\'d crumbled into I might\'ve let them lay [Chorus: Taylor Swift] Are you really gonna talk about timing in times like these? And let all your damage damage me And carry your baggage up my street And make me your future history, it\'s time You\'ve come a long way; open the blinds, let me see your face You wouldn\'t be the first renegade to need somebody [Outro: Taylor Swift & Justin Vernon] To need somebody, to need somebody, to need somebody, to need Are you really gonna talk about timing in times like these? And let all your damage damage me And carry your baggage up my street And make me your future history, it\'s time You\'ve come a long way; open the blinds, let me see your face You wouldn\'t be the first renegade to need somebody Is it insensitive for me to say, "Get your shit together so I can love you"? You\'ve come a long way, open the blinds, let me see your face You wouldn\'t be the first renegade to need somebody Is it really your anxiety that stops you from giving me everything? Or do you just not want to?',
    Length: '04:14',
    Genre: 'hip hop',
    Album: 'How Long Do You Think It\'s Gonna Last?',
    Release_Date: '2021-08-27',
    Popularity: '66',
    Feature_Scores: {
      Energy: '71',
      Danceability: '53',
      Positiveness: '58',
      Speechiness: '5',
      Liveness: '9',
      Acousticness: '44',
      Instrumentalness: '2'
    },
    Contextual_Tags: [ 'Good for Party', 'Good for Exercise', 'Good for Running' ],
    Similar_Songs: [
      {
        Artist: 'Melanie Martinez',
        Title: 'Glued',
        Similarity_Score: 0.9565916787
      },
      {
        Artist: 'ElGrandeToto',
        Title: 'ElGrandeToto - Rhghayer',
        Similarity_Score: 0.9522145914
      },
      {
        Artist: 'Melanie Martinez',
        Title: 'Alphabet Bitch',
        Similarity_Score: 0.9511421673
      }
    ]
  }
]
spotify_db>
```